C++ Standard Library Issues Resolved Directly In Issaquah

Doc. no. P0304R1

Date: Revised 2016-11-11 at 22:11:52 UTC

Project: Programming Language C++
Reply to: Marshall Clow <lwgchair@gmail.com>

Immediate Issues

2770. `tuple_size<const T>` specialization is not SFINAE compatible and breaks decomposition declarations

Section: 20.5.2.6 [tuple.helper] **Status:** Immediate **Submitter:** Richard Smith **Opened:** 2016-08-15 **Last modified:** 2016-11-11

Priority: 1

View all other issues in [tuple.helper].

Discussion:

Consider:

```
#include <utility>

struct X { int a, b; };
const auto [x, y] = X();
```

This is ill-formed: it triggers the instantiation of `std::tuple_size<const X>`, which hits a hard error because it tries to use `tuple_size<X>::value`, which does not exist. The code compiles if `<utility>` (or another header providing `tuple_size`) is not included.

It seems that we either need a different strategy for decomposition declarations, or we need to change the library to make the `tuple_size` partial specializations for *cv*-qualifiers SFINAE-compatible.

The latter seems like the better option to me, and like a good idea regardless of decomposition declarations.

[2016-09-05, Daniel comments]

This is partially related to LWG 2446.

[2016-09-09 Issues Resolution Telecom]

Geoffrey to provide wording

[2016-09-14 Geoffrey provides wording]

[2016-10 Telecom]

Alisdair to add his concerns here before Issaquah. Revisit then. Status to 'Open'

Proposed resolution:

This wording is relative to N4606.

1. Edit 20.5.2.6 [tuple.helper] as follows:

```
template <class T> class tuple_size<const T>;
template <class T> class tuple_size<volatile T>;
template <class T> class tuple_size<const volatile T>;
```

-4- Let *TS* denote `tuple_size<T>` of the *cv*-unqualified type `T`. If the expression *TS*`::value` is well-formed when treated as an unevaluated operand, t~~T~~hen each of the three templates shall meet the `UnaryTypeTrait` requirements (20.15.1) with a `BaseCharacteristic` of

```
integral_constant<size_t, TS::value>
```

Otherwise, they shall have no member `value`.

Access checking is performed as if in a context unrelated to *TS* and `T`. Only the validity of the immediate context of the expression is considered. [*Note:* The compilation of the expression can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the "immediate context" and can result in the program being ill-formed. — *end note*]

-5- In addition to being available via inclusion of the `<tuple>` header, the three templates are available when either of the headers `<array>` or `<utility>` are included.